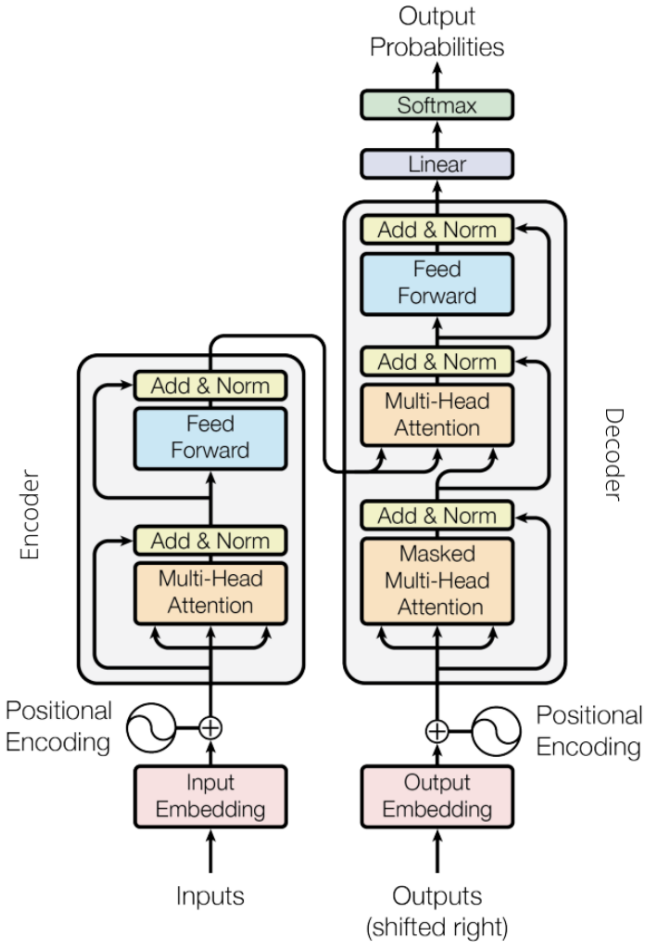
ECE 590
Neural-Network-Based Large Language Models

Tuesdays and Thursdays 1:25-2:40
Fitzpatrick Schiciano A 1464

ECE 590
Neural-Network-Based Large Language Models

Lecture 19
03/26/2026


Output
Probabilities
Softmax
Linear
Add & Norm
Feed
Forward
Add & Norm
Multi-Head
Attention
Add & Norm
Masked
Multi-Head
Attention
Decoder
Encoder
Add & Norm
Feed
Forward
Add & Norm
Multi-Head
Attention
Positional
Encoding
Positional
Encoding
Input
Embedding
Output
Embedding
Inputs
Outputs
(shifted right)

Feedforward Elements Within Transformer

- The feedforward element plays an important role in both the Transformer

- May express a typical FF element as

$$FF_\theta(x) = Bf[Ax + a] + b$$

where A and B are matrices, a and b are vectors (biases), and $f(\cdot)$ is a pointwise nonlinearity

- In language models, the number of tokens could be $> 100{,}000$, each with its own learned embedding vector, meaning there could be significant diversity in x

Feedforward Elements Within Transformer

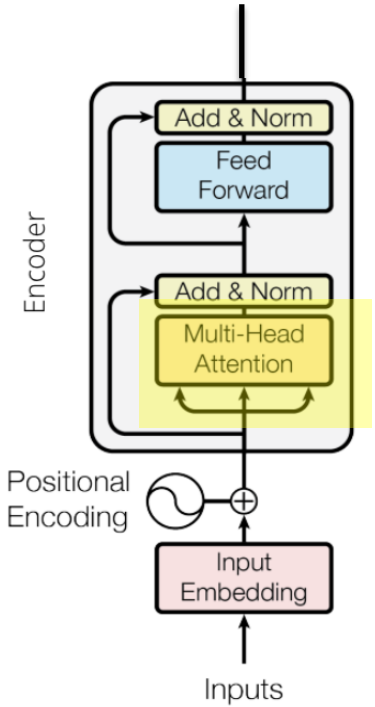
$$FF_\theta(x) = Bf[Ax + a] + b$$

- In language models, the number of tokens could be $> 100{,}000$, each with its own learned embedding vector, meaning there could be significant diversity in x

- To cover the full breadth and diversity of x , the size of the output dimension of hidden vector $Ax + b$ will likely have to be enormous

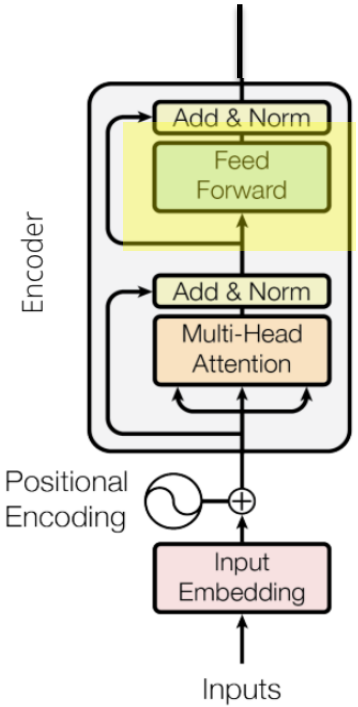
- Computationally expensive in the forward pass of the Transformer, despite the fact that only a small portion of the hidden vector space will be relevant for any particular x

Scaling Up the Transformer



- We have discussed scaling of the attention layers for handling long context
- Efficient computation of attention kernels via randomization, yielding attention calculations that are linear in the length of the token sequence
- Also considered the Nystrom method, in which we employ a small number of landmark keys and queries, and extrapolate from those

Scaling Up the Transformer



- The feedforward element plays an important role in the Transformer, which we have examined from the standpoint of functional GD
- As the token vocabulary and breadth of context increases, the variety in the vectors input to the FF element increases
- Challenging for one FF element to handle all that diversity, motivating multiple specialized FF elements
- Significantly accelerates inference computations

Mixture of Experts

$$FF_\theta(x) = Bf[Ax + a] + b$$

- Rather than building one massive FF element, we can design many smaller "expert" FF elements that specialize on particular types of input vectors
- A "gating network" takes in the vector x and ascertains which experts are most relevant
- The gating network leads to only a small set of experts to be applied to any input x
- This yields significant acceleration of the forward pass of the Transformer, allowing scaling to analysis of massive corpora

History

$$FF_\theta(x) = Bf[Ax + a] + b$$

- MoE models for FF elements have been the subject of extensive research, dating to the early 1990s

  R.A. Jacobs, M.I. Jordan, S.J. Nowlan, and G.E. Hinton. "Adaptive mixtures of local experts," Neural Computing, 1991.

- A resurgence of interest in MoE models realized recently, as Transformer models have scaled to "internet scale"

- Important for computational efficiency of inference when operating at scale

MoE Formulation: Gating Network

- Let the input to the MoE be represented as $x \in R^d$

- Let $G(x)$ be a gating network, that has an output the weights the importance/relevance of each of n experts

- For large-scale language models, there could be $n > 1000$ experts, and for any particular input x only 2 or 3 experts will be used

- Therefore, the gating network typically has a <u>sparse</u> n-dimensional output

- One of the challenges is that the experts typically distributed across multiple GPUs (load balancing, communication balancing)

Experts

- Let $E_i(x)$ represent the ith expert, with input x

- There are $i = 1, \ldots, n$ distinct experts (distinct FF elements)

- We desire that each expert be a relatively simple, specialized neural network

$$E_i(x) = B_i f[A_i x + a_i] + b_i$$

where the number of rows of A_i (number of hidden units) is relatively small

MoE vs One Large FF Element

- For n experts, we have n models of the form

$$E_i(x) = B_i f[A_i x + a_i] + b_i$$

- The total number of parameters $\{A_i, B_i, a_i, b_i\}_{i=1,n}$ is <u>the</u> <u>same</u> as would be used in one massive FF element

- The amount of needed training data is unchanged

- The advantage comes in the forward pass of the model (which we do very often), for which with the MoE setup only a small number of experts are used for any input x

Summary of MoE Model

- The output from an MoE unit may be expressed as

$$y = \sum_{i=1}^{n} G_i(x) E_i(x)$$

where $G_i(x)$ is the output from the ith component of the gating network

- When $G_i(x) = 0$, the associated expert $E_i(x)$ need not be computed

- Very few experts are "on" for any input x; $G(x)$ very sparse

Design of Gating Network

$$y = \sum_{i=1}^{n} G_i(x) E_i(x)$$

- The simplest (and first) form of the gating network was

$$G_\sigma(x) = \mathrm{Softmax}(W_g x)$$

- W_g is a learnable matrix, with n rows, for the n experts

- Need to introduce some sort of cutoff on the softmax output to impose the desired sparsity

Top-K Gating

- Only retain the top-k most probable outputs from the softmax:

$$G(x) = \text{Softmax}[\text{KeepTopK}(H(x), k)]$$

where

$$[\text{KeepTopK}(v, k)]_i = \begin{cases} v_i, & \text{if } v_i \text{ is among largest } k \text{ elements of } v \\ -\infty, & \text{otherwise} \end{cases}$$

- Turns "off" all but the k largest elements, yielding sparsity in the gating-network output

Noisy Top-K Gating

$$G(x) = \text{Softmax}[\text{KeepTopK}(H(x), k)]$$

$$[\text{KeepTopK}(v, k)]_i = \begin{cases} v_i, & \text{if } v_i \text{ is among largest } k \text{ elements of } v \\ -\infty, & \text{otherwise} \end{cases}$$

- Introduce a *noisy* form for $H(x)$, as

$$H_i(x) = \left(W_g x\right)_i + \epsilon_i \cdot \text{Softplus}[(W_{noise} x)_i]$$

where $\epsilon_i \sim N(0,1)$ and

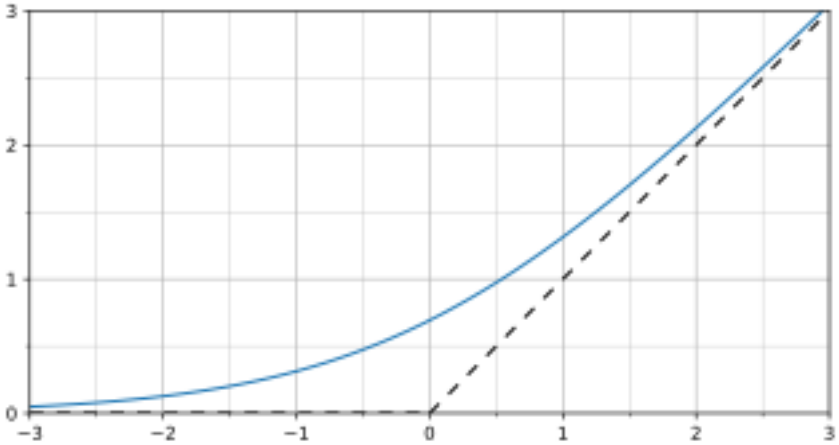
$$\text{Softplus}(\alpha) = \log[1 + e^{\alpha}]$$

Softplus

$$H_i(x) = \left(W_g x\right)_i + \epsilon_i \cdot \mathrm{Softplus}[(W_{noise} x)_i]$$

$$\mathrm{Softplus}(\alpha) = \log[1 + e^x]$$

- The softplus is a smoothed (differentiable everywhere) form of ReLu



Noisy Inputs to Softmax

$$H_i(x) = \left(W_g x\right)_i + \underbrace{\epsilon_i \cdot \mathrm{Softplus}[(W_{noise} x)_i]}$$

Input-dependent noise

Noise of each component is zero-mean Gaussian with standard deviation $\mathrm{Softplus}[(W_{noise} x)_i]$

- The noise manifests stochasticity in the assignment of experts for given input x

- This has been found to improve training and enhance load balancing across the n experts (avoids repeated use of the same small set of experts)

Challenges When Training

$$G(x) = \text{Softmax}[\text{KeepTopK}(H(x), k)]$$

$$H_i(x) = \left(W_g x\right)_i + \epsilon_i \cdot \text{Softplus}[(W_{noise} x)_i]$$

- It has been found that when training an MoE model, there is a tendency for the training to yield a model for which only a small set of experts are used repeatedly
- This can significantly undermine model performance, as each expert is relatively small, and we do not achieve the desired expert specialization
- The noise term helps with balancing use of experts, but not enough by itself

Balancing Expert Utilization with Additional Loss

- It is necessary to impose additional terms to the loss function, in addition to the cross-entropy fit to language, to assure balanced use of all experts

- Let X represent the set of vectors connected with a given mini-batch while training

- Define the term

$$\text{Importance}(X) = \frac{1}{|X|} \sum_{x \in X} G(x)$$

- This is a vector, defining the average weighting across all samples in X, where the weight is manifested across all n components of the weighting vector

Balancing Expert Utilization with Additional Loss

$$\text{Importance}(X) = \frac{1}{|X|} \sum_{x \in X} G(x)$$

- Need a measure connected to $\text{Importance}(X)$, which should reward that it is uniformly spread across the n experts

- Coefficient of variation:

$$CV(V) = \frac{\sigma}{\mu}$$

- Minimized if V is a uniform vector, as constant weights across components yields zero variance

First Additional Term

$$\text{Importance}(X) = \frac{1}{|X|}\sum_{x \in X} G(x) \qquad CV(V) = \frac{\sigma}{\mu}$$

- Loss term on the model parameters θ modified as

$$L(\theta; Data) = L_{cross\ entropy}(\theta; Data) + \lambda_1 CV(\text{Importance}(X))$$

- Seek model parameters that minimize composite loss, and $\lambda_1 > 0$ is a parameter that needs to be set

Issues with this Term

$$\text{Importance}(X) = \frac{1}{|X|}\sum_{x \in X} G(x) \qquad CV(V) = \frac{\sigma}{\mu}$$

$$L(\theta; Data) = L_{cross\ entropy}(\theta; Data) + \lambda_1 CV(\text{Importance}(X))$$

- This term is good, but is doesn't address all issues of balance

- A small subset of components of $G(x)$ could have very large values for a small subset of X, while other components of $G(x)$ may have small values for a large number of samples in X

Issues with this Term

$$\text{Importance}(X) = \frac{1}{|X|}\sum_{x \in X} G(x) \qquad CV(V) = \frac{\sigma}{\mu}$$

$$L(\theta; Data) = L_{cross\ entropy}(\theta; Data) + \lambda_1 CV(\text{Importance}(X))$$

- Google included an additional term to enhance load balance
- More recently DeepSeek developed a method for balancing that is simpler, and we will concentrate on it

More Details on Transformer Implementation from DeepSeek

- The lth attention block in a conventional Transformer may be expressed as

$$u_{1:T}^{l} = \text{Self_Att}\left(h_{1:T}^{l-1}\right) + h_{1:T}^{l-1}$$

$$h_t^l = FFN\left(u_t^l\right) + u_t^l$$

- With an MoE setup, we generalize the FFN as

$$h_t^l = \sum_{i=1}^{n} g_{i,t} FFN_i\left(u_t^l\right) + u_t^l$$

$$g_t = \underbrace{\text{Softmax}[\text{KeepTopK}\left(W_g u_t^l, k\right)]}_{\text{Simplified, no additive noise}}$$

Specialize the Discussion for the Transformer

- The lth attention block in a conventional Transformer may be expressed as

$$u_{1:T}^{l} = \text{Self_Att}\left(h_{1:T}^{l-1}\right) + h_{1:T}^{l-1}$$

$$h_t^l = FFN\left(u_t^l\right) + u_t^l$$

- With an MoE setup, we generalize the FFN as

$$h_t^l = \sum_{i=1}^{n} g_{i,t} FFN_i\left(u_t^l\right) + u_t^l$$

ith component of g_t

$$g_t = \underbrace{\text{Softmax}[\text{KeepTopK}\left(W_g u_t^l, k\right)]}_{\text{Simplified, no additive noise}}$$

DeepSeek Modifications to MoE on FF Elements

$$h_t^l = \sum_{i=1}^{mn} g_{i,t} FFN_i\left(u_t^l\right) + u_t^l \qquad g_t = \mathrm{Softmax}[\mathrm{KeepTopK}\left(W_g u_t^l, mk\right)]$$

- Seek to manifest more specialized experts, with more of them used for each sample u_t^l
- Previously we had n experts, now we'll have mn experts, with $m > 1$
- Each expert will have $1/m$ the parameters, so total parameters unchanged
- Also seek less sparse weights, with top-k going to top mk

More Important Change: Shared FF Elements

$$h_t^l = \sum_{i=1}^{K_s} FFN_i(u_t^l) + \sum_{i=K_s+1}^{mn} g_{i,t} FFN_i(u_t^l) + u_t^l$$

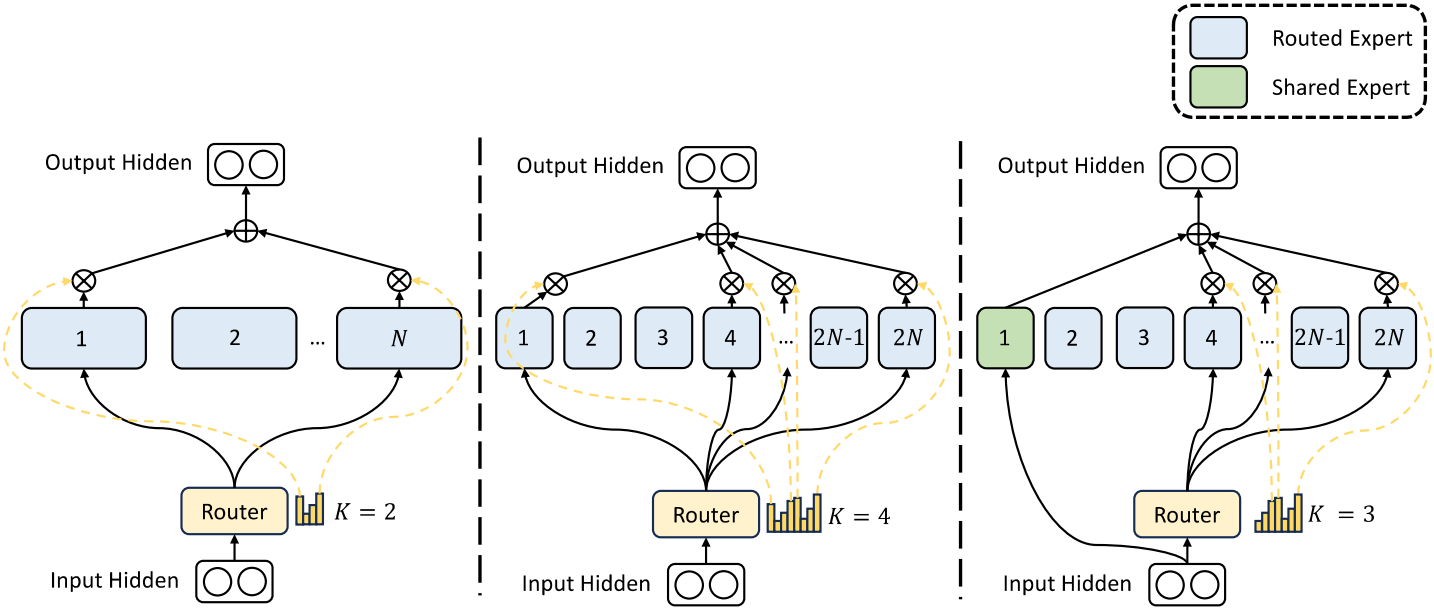
- Impose that there are K_s FF elements that are used by all samples
- These can be viewed as a general-purpose FF elements
- The MoE experts used to capture specialized FF behavior

More Important Change: General-Purpose FFNs

Gating weights with $mn - K_s$ components

$$h_t^l = \underbrace{\sum_{i=1}^{K_s} FFN_i(u_t^l)}_{K_s \text{ shared FFN}} + \underbrace{\sum_{i=K_s+1}^{mn} g_{i,t} FFN_i(u_t^l)}_{mn - K_s \text{ MoE FFN}} + u_t^l$$

DeepSeek MoE



(a) Conventional Top-2 Routing → **(b) + Fine-grained Expert Segmentation** → **(c) + Shared Expert Isolation (DeepSeekMoE)**

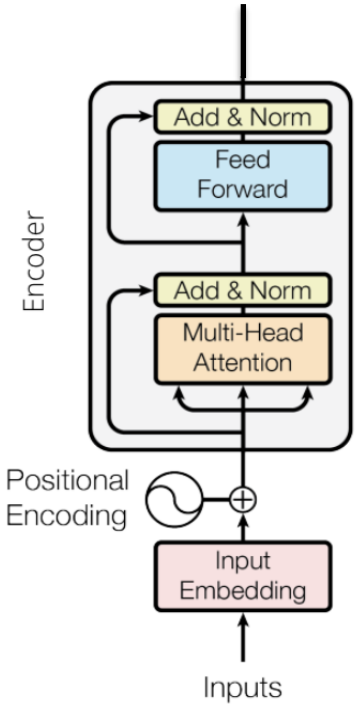
Load-Balancing Augmentation of Cost Function

- Like in the discussion above, it is essential to add loss terms that encourage distributed and specialized use of the MoE FFN experts
- DeepSeek used a term like that discussed previously, to encourage widely distributed gating networks
- Details a little different from earlier, but similar
- Deepseek also encouraged distributed use of experts across compute devices, and introduce a (simple) added loss term to encourage that

Importance of MoE FFNs for Large-Scale Transformers

- While the total number of FF parameters is the same with and without MoE, the number of parameters in use for any given sample markedly reduced
- This accelerates inference significantly
- Significant care must be taken to assure diversity of expert usage, and to distribute experts across different machines (parallel implementation)
- The MoE setup usually used on each layer of FF networks, but not in the attention layers

Applications of the Transformer Beyond Language



- We have developed each of the pieces of the Transformer and have sought to get a sense of their roles
- Have examined training methods (Adam) and alignment for particular objectives (RL: PPO & GRPO)
- Have examined how the Transformer can be scaled to massive corpora, accelerating the attention mechanism and the FF element
- There are increasingly many applications of the Transformer beyond language: 2024 Nobel Prize in Chemistry